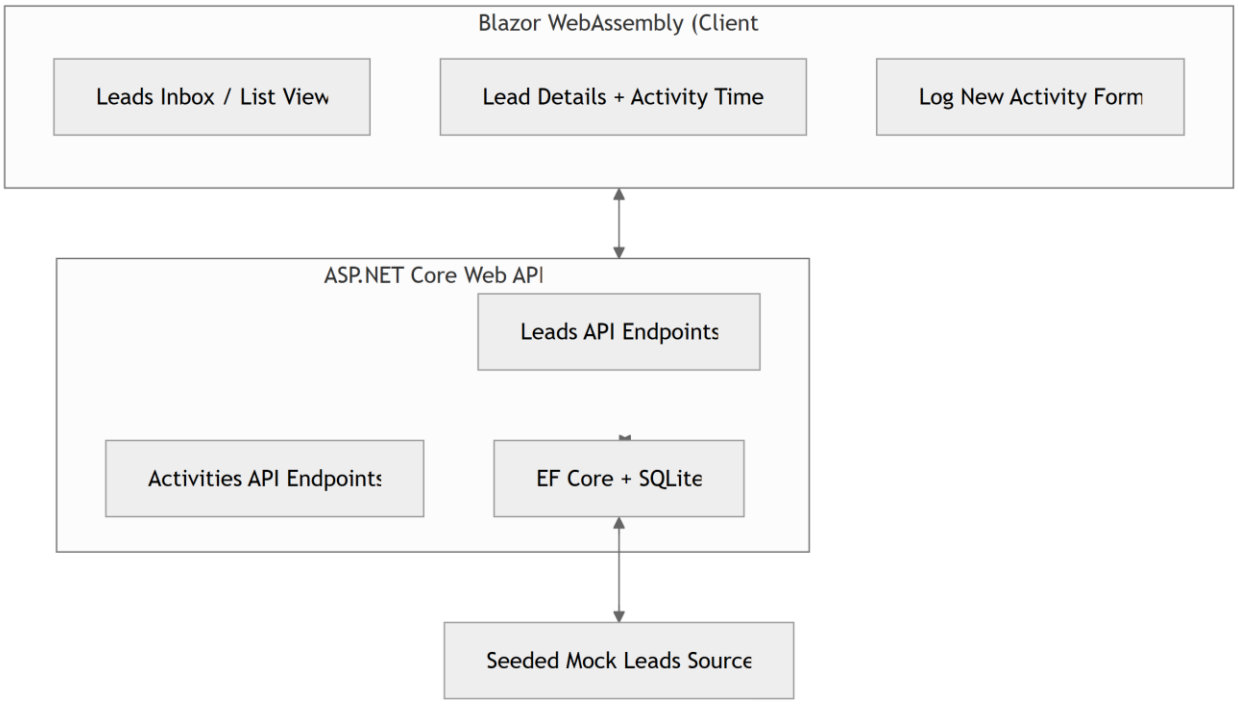
System Design Document - Sales Lead Management Tool (Scenario C)

Date: June 16, 2026

Chosen Scenario: C - The Sales Lead Management Tool

Chosen Stack: Latest .NET ecosystem

1. Architecture Diagram (Text-based / Mermaid-ready)



ASP.NET Core Web API

Blazor WebAssembly (Client-side)

Leads Inbox / List View

Lead Details + Activity Timeline

Log New Activity Form

Leads API Endpoints

Activities API Endpoints

EF Core + SQLite

Seeded Mock Leads Source

2. Component Roles

- **Blazor Frontend**: Responsive single-page application for lead management, listing, details, and activity logging.
- **ASP.NET Core Web API**: Handles all business logic, data operations, and exposes clean REST endpoints.
- **Entity Framework Core + SQLite**: Persistent storage with minimal setup.
- **Mock Lead Source**: Startup data seeding to simulate incoming website leads.

3. Data Flow

1. Blazor app loads → Calls GET /api/leads to populate the inbox.
2. User clicks a lead → GET /api/leads/{id} fetches details + chronological activities.
3. User submits new activity → POST /api/leads/{id}/activities persists the record and refreshes the UI.
4. All interactions are logged and validated on the backend.

Key Entities:

- **Lead**: Id, Name, Email, Phone, VehicleInterest, Status, Source, CreatedDate, etc.
- **LeadActivity**: Id, LeadId (FK), Type (Call, Email, Meeting, Note, etc.), Notes, Timestamp, PerformedBy.

4. Chosen Technologies & Justifications (Latest Versions)

- **Backend**: **.NET 10.0** (LTS) + ASP.NET Core Web API → Current latest Long Term Support version (released Nov 2025, actively maintained). Excellent performance, built-in OpenAPI, minimal APIs, and modern C# features.
- **Frontend**: **Blazor WebAssembly** (latest .NET 10 templates) → Full C#/.NET stack for UI, strong component model, and seamless integration with the backend.
- **Database**: SQLite + **Entity Framework Core** (latest) → Zero-config, cross-platform, file-based persistence perfect for this assessment.
- **Other Tools**:
    - Serilog (structured logging)
    - FluentValidation

    - xUnit + bUnit (testing)
    - MudBlazor or Tailwind for UI styling (modern and lightweight)

Justification: Using the absolute latest stable LTS versions (.NET 10) demonstrates up-to-date knowledge, access to the newest performance improvements, security features, and developer experience enhancements, while keeping the solution simple, maintainable, and production-oriented.

5. Observability Strategy

- **Logging**: Serilog with console/file sinks for structured logging of requests, business events, and errors.
- **Health Checks**: Built-in ASP.NET Core Health Checks middleware (/health).
- **Metrics & Tracing**: Correlation IDs + basic request metrics (easily extensible to OpenTelemetry / Application Insights).
- **Error Handling**: Global exception filter with consistent error responses.

6. GenAI Collaboration in Design Phase

I worked with Grok (xAI) to:

- Confirm the latest .NET 10 stack and its benefits.
- Generate initial EF Core models, DbContext, and repository patterns.
- Design clean REST API endpoints and Blazor component structure.
- Review best practices for observability and testing in .NET 10.

All outputs were carefully reviewed, refined, and validated for quality and alignment with the requirements.